



本科实验报告

课程名称:	计算机网络
姓 名:	刘仁钦
学 院:	计算机科学与技术学院
专 业:	计算机科学与技术
学 号:	3230106230
指导教师:	韩劲松

2025 年 12 月 06 日

目录

一、 实验目的	3
二、 实验内容	3
三、 主要仪器设备	3
四、 操作方法与实验步骤	3
1. 实现序列号包装和解包装	3
1.1. 在 libsponge/wrapping_integers.cc 中实现 wrap 和 unwrap 函数	4
2. 实现 TCP 接收方	4
2.1. 在 libsponge/tcp_receiver.hh 中添加必要的成员变量	4
2.2. 在 libsponge/tcp_receiver.cc 中实现三个方法	4
3. 实现 TCP 发送方	5
3.1. 在 libsponge/tcp_sender.hh 中添加必要的成员变量	5
3.2. 在 libsponge/tcp_sender.cc 中实现核心方法	6
五、 实验数据记录和处理	7
1. 实验结果	7
1.1. 问题一	7
1.2. 问题二	7
2. 思考题	8
2.1. 问题一	8
2.2. 问题二	9
2.3. 问题三	10
2.4. 问题四	10
六、 讨论、心得	11
七、 附录	11
1. libsponge/wrapping_integers.cc	11
2. libsponge/tcp_receiver.hh	13
3. libsponge/tcp_receiver.cc	15
4. libsponge/tcp_sender.hh	17
5. libsponge/tcp_sender.cc	20

浙江大学实验报告

实验项目名称: Lab4 TCP 接收方 (Receiver) 与发送方 (Sender)

学生姓名: 刘仁钦 专业: 计算机科学与技术 学号: 3230106230

一、实验目的

- 学习掌握 TCP 的工作原理
- 学习掌握流重组器的工作原理
- 学习掌握 TCP receiver 和 TCP sender 的相关知识

二、实验内容

- 实现流重组器, 一个将字节流的字串或小段按照正确顺序来拼接回连续字节流的模块。
- 实现 TCPReceiver。
 - 接收 TCPsegment;
 - 重新组装字节流;
 - 确定应该发回发送者的信号, 以进行数据确认和流量控制。
- 实现 TCPSender:
 - 将 ByteStream 中的数据以 TCP 报文形式持续发送给接收者;
 - 处理 TCPReceiver 传入的 ackno 和 window size, 以追踪接收者当前的接收状态, 以及检测丢包的情况;
 - 若经过一个超时时间仍然没有接收到 TCPReceiver 发送的 ack 包, 则重传。
 - 发送一个空段

三、主要仪器设备

- 联网的 PC 机
- Linux 虚拟机

四、操作方法与实验步骤

本次实验的核心任务分为三个阶段。首先是实现序列号包装和解包装函数, 用于在 32 位循环序列号和 64 位绝对序列号之间进行转换。其次是实现 TCP 接收方, 负责接收 TCP 段、重组字节流并计算确认号和窗口大小。最后是实现 TCP 发送方, 负责将数据分段发送、处理确认和超时重传。

1. 实现序列号包装和解包装

TCP 协议使用 32 位循环序列号, 但在内部处理时需要使用 64 位绝对序列号。

`wrapping_integers` 模块提供了两者之间的转换功能。

1.1. 在 `libsponge/wrapping_integers.cc` 中实现 `wrap` 和 `unwrap` 函数

`wrap` 函数将 64 位绝对序列号转换为 32 位循环序列号。实现较为简单，只需要将绝对序列号与初始序列号 (ISN) 相加，然后取模 2^{32} 即可。

`unwrap` 函数将 32 位循环序列号转换为 64 位绝对序列号，这是本次实验的难点之一。由于 32 位序列号会循环，我们需要找到最接近 checkpoint 的绝对序列号。实现思路如下：

1. 计算偏移量： `offset = n - isn` (32 位无符号减法)，这是结果的低 32 位
2. 利用 checkpoint 确定周期：将 checkpoint 的高 32 位与 offset 拼接，得到候选值 `t = (checkpoint & 0xFFFFFFFF00000000) | offset`，这样 `t` 和 `checkpoint` 在一个 2^{32} 周期内
3. 判断是否需要调整到相邻周期：
 - 如果 `t > checkpoint` 且差距超过半个周期 (2^{31})，说明应该使用上一个周期的值，即 `t - 2^32` (但要防止 `uint64` 下溢)
 - 如果 `t < checkpoint` 且差距超过半个周期，说明应该使用下一个周期的值，即 `t + 2^32`
4. 返回最接近 checkpoint 的值。

具体代码参见附录。

2. 实现 TCP 接收方

TCP 接收方负责接收 TCP 段，将数据重组为连续的字节流，并计算确认号和窗口大小反馈给发送方。

2.1. 在 `libsponge/tcp_receiver.hh` 中添加必要的成员变量

为了实现 TCP 接收方的功能，我们需要添加以下成员变量：

- `_isn`：存储从远程发送方接收到的初始序列号 (ISN)，使用 `std::optional` 表示是否已收到 SYN
- `_fin_received`：标记是否已收到 FIN 标志

具体代码参见附录

2.2. 在 `libsponge/tcp_receiver.cc` 中实现三个方法

1. `segment_received(seg)`：处理接收到的 TCP 段
 1. 首先检查是否处于 Listen 状态 (`_isn` 为空)。如果是，则检查是否为 SYN 包。只有收到 SYN 包才能建立连接，否则丢弃数据包。

2. 对于已建立连接的情况，需要将 TCP 段的 payload 数据放入流重组器。关键步骤包括：
 - 使用 `unwrap` 将 32 位序列号转换为 64 位绝对序列号
 - 将绝对序列号转换为流索引（排除 SYN 和 FIN）
 - 对于 SYN 包，payload 从绝对序列号 1 开始（流索引 0）
 - 对于普通包，流索引 = 绝对序列号 - 1
 - 调用 `_reassembler.push_substring` 将数据放入流重组器
3. 如果收到 FIN 标志，标记 `_fin_received` 为 true
2. `ackno()`：计算并返回确认号
 1. 如果尚未收到 SYN（处于 Listen 状态），返回空
 2. 否则，查询流重组器中已写入的字节数，将其转换为绝对序列号（需要加上 SYN 占用的 1 个序列号）
 3. 如果已收到 FIN 且流已结束，还需要加上 FIN 占用的 1 个序列号
 4. 最后使用 `wrap` 将绝对序列号转换为 32 位序列号返回
3. `window_size()`：计算窗口大小
 1. 窗口大小 = 总容量 - 流重组器中已存数据量
 2. 已存数据量 = 已写入字节数 - 已读取字节数

具体代码参见附录

3. 实现 TCP 发送方

TCP 发送方负责将 `ByteStream` 中的数据分段发送，处理接收方的确认和窗口大小，并实现超时重传机制。

3.1. 在 `libsponge/tcp_sender.hh` 中添加必要的成员变量

为了实现 TCP 发送方的功能，我们需要添加以下成员变量：

- `_ackno`：已确认的绝对序列号
- `_window_size`：远程接收方通告的窗口大小
- `_outstanding_segments`：一个 `std::map`，用于存储已发送但尚未确认的段（键为段的起始绝对序列号）
- `_rto`：当前的重传超时时间（RTO）
- `_timer`：重传计时器
- `_timer_running`：计时器是否正在运行

- `_consecutive_retransmissions` : 连续重传计数
- `_syn_sent` 和 `_fin_sent` : 标记是否已发送 SYN 和 FIN

具体代码参见附录

3.2. 在 `libsponge/tcp_sender.cc` 中实现核心方法

1. `fill_window()` : 填充窗口, 尽可能多地发送数据
 1. 计算窗口右边界: `window_right = _ackno + effective_window` (如果窗口大小为 0, 视为 1)
 2. 循环发送, 直到 `_next_seqno ≥ window_right` :
 - 如果尚未发送 SYN, 发送 SYN 段
 - 计算可发送的 payload 大小 (不超过窗口大小、MAX_PAYLOAD_SIZE 和可用数据量)
 - 如果满足条件, 添加 FIN 标志
 - 发送段并更新 `_next_seqno`
 - 如果设置了 FIN, 退出循环
2. `ack_received(ackno, window_size)` : 处理接收到的确认
 1. 将 `ackno` 转换为绝对序列号
 2. 如果 `ackno` 不可靠 (大于 `_next_seqno`), 直接丢弃
 3. 更新窗口大小
 4. 遍历 `_outstanding_segments`, 移除已被确认的段
 5. 如果确认了新数据, 重置计时器和 RTO, 重置连续重传计数
 6. 更新 `_ackno`
 7. 调用 `fill_window()` 继续发送数据
3. `tick(ms_since_last_tick)` : 处理时间流逝
 1. 如果计时器正在运行, 更新计时器
 2. 如果计时器超时, 重传序列号最小的未确认段
 3. 如果窗口大小不为 0, 将 RTO 加倍 (指数退避)
 4. 增加连续重传计数
4. `send_empty_segment()` : 发送空段
 1. 创建一个 payload 为空的 TCP 段, 设置正确的序列号
5. `bytes_in_flight()` : 计算正在传输的字节数

1. 遍历 `_outstanding_segments` , 累加所有段的 `length_in_sequence_space()`

具体代码参见附录

五、实验数据记录和处理

1. 实验结果

1.1. 问题一

运行 `ctest -R wrap` 命令的测试结果展示

```
1 cd zju-comnet-labs/build bash
2 ctest -R wrap
```

运行测试命令，测试结果如图 1。

```
.../zju-comnet-labs/build main • ? } ctest -R wrap
Test project /home/cyrus28214/courses/Net/zju-comnet-labs/build
  Start 1: t_wrapping_ints_cmp
1/4 Test #1: t_wrapping_ints_cmp ..... Passed    0.00 sec
  Start 2: t_wrapping_ints_unwrap
2/4 Test #2: t_wrapping_ints_unwrap ..... Passed    0.00 sec
  Start 3: t_wrapping_ints_wrap
3/4 Test #3: t_wrapping_ints_wrap ..... Passed    0.00 sec
  Start 4: t_wrapping_ints_roundtrip
4/4 Test #4: t_wrapping_ints_roundtrip ..... Passed    0.07 sec

100% tests passed, 0 tests failed out of 4

Total Test time (real) = 0.08 sec
```

图 1: 运行 `ctest -R wrap` 命令的测试结果

1.2. 问题二

运行 `make check_lab2` 命令的测试结果展示

```
1 cd zju-comnet-labs/build bash
2 make check_lab2
```

运行测试命令，测试结果如图 2。

```

      Start 17: t_send_extra
17/34 Test #17: t_send_extra ..... Passed    0.00 sec
      Start 18: t_strm_reassem_single
18/34 Test #18: t_strm_reassem_single ..... Passed    0.00 sec
      Start 19: t_strm_reassem_seq
19/34 Test #19: t_strm_reassem_seq ..... Passed    0.00 sec
      Start 20: t_strm_reassem_dup
20/34 Test #20: t_strm_reassem_dup ..... Passed    0.00 sec
      Start 21: t_strm_reassem_holes
21/34 Test #21: t_strm_reassem_holes ..... Passed    0.00 sec
      Start 22: t_strm_reassem_many
22/34 Test #22: t_strm_reassem_many ..... Passed    0.02 sec
      Start 23: t_strm_reassem_overlapping
23/34 Test #23: t_strm_reassem_overlapping ..... Passed    0.00 sec
      Start 24: t_strm_reassem_win
24/34 Test #24: t_strm_reassem_win ..... Passed    0.02 sec
      Start 25: t_strm_reassem_cap
25/34 Test #25: t_strm_reassem_cap ..... Passed    0.03 sec
      Start 26: t_byte_stream_construction
26/34 Test #26: t_byte_stream_construction ..... Passed    0.00 sec
      Start 27: t_byte_stream_one_write
27/34 Test #27: t_byte_stream_one_write ..... Passed    0.00 sec
      Start 28: t_byte_stream_two_writes
28/34 Test #28: t_byte_stream_two_writes ..... Passed    0.00 sec
      Start 29: t_byte_stream_capacity
29/34 Test #29: t_byte_stream_capacity ..... Passed    0.18 sec
      Start 30: t_byte_stream_many_writes
30/34 Test #30: t_byte_stream_many_writes ..... Passed    0.00 sec
      Start 31: t_webget
31/34 Test #31: t_webget ..... Passed    1.05 sec
      Start 53: t_address_dt
32/34 Test #53: t_address_dt ..... Passed    0.01 sec
      Start 54: t_parser_dt
33/34 Test #54: t_parser_dt ..... Passed    0.00 sec
      Start 55: t_socket_dt
34/34 Test #55: t_socket_dt ..... Passed    0.01 sec

100% tests passed, 0 tests failed out of 34

Total Test time (real) = 1.51 sec
[100%] Built target check_lab2

```

图 2: 运行 `make check_lab2` 命令的测试结果

2. 思考题

2.1. 问题一

通过代码，请描述 TCPSender 是如何发送出一个 segment 的？

TCPSender 通过 `fill_window()` 方法发送 segment，具体流程如下：

1. 首先计算窗口右边界: `window_right = _ackno + effective_window` (如果窗口大小为 0, 视为 1)
2. 循环发送, 直到 `_next_seqno ≥ window_right`:
 - 如果尚未发送 SYN, 创建 SYN 段并发送
 - 否则, 计算可发送的 payload 大小 (不超过窗口大小、MAX_PAYLOAD_SIZE 和可用数据量)
 - 从 `_stream` 中读取数据, 创建 TCP 段
 - 如果满足条件 (流已结束且窗口有空间), 添加 FIN 标志
 - 设置段的序列号 (使用 `wrap` 将绝对序列号转换为 32 位序列号)
 - 将段加入 `_outstanding_segments` 进行追踪
 - 将段推入 `_segments_out` 队列等待发送
 - 更新 `_next_seqno`
 - 如果这是第一个待发送的段, 启动重传计时器
 - 如果设置了 FIN, 退出循环
3. 发送的段会被 TCPConnection 层取出, 添加接收方的 ackno 和 window size 后发送到网络。

2.2. 问题二

请用自己的理解描述一下 TCPSender 超时重传的整个流程。

TCPSender 的超时重传机制通过 `tick()` 方法实现, 整个流程如下:

1. 计时器管理: 当发送第一个段时, 启动重传计时器 (`_timer_running = true`, `_timer = 0`)
2. 时间更新: 系统周期性调用 `tick(ms_since_last_tick)`, 更新计时器:
`_timer += ms_since_last_tick`
3. 超时检测: 如果 `_timer ≥ _rto`, 说明超时
4. 重传处理:
 - 重置计时器: `_timer = 0`
 - 重传序列号最小的未确认段 (`_outstanding_segments` 中的第一个)
 - 如果窗口大小不为 0, 将 RTO 加倍 (指数退避): `_rto *= 2`
 - 增加连续重传计数: `_consecutive_retransmissions++`

5. 确认处理：当收到 ACK 确认时（`ack_received`）：
 - 如果确认了新数据，重置计时器：`_timer = 0`
 - 重置 RTO 为初始值：`_rto = _initial_retransmission_timeout`
 - 重置连续重传计数：`_consecutive_retransmissions = 0`
6. 停止计时器：如果所有数据都被确认（`_outstanding_segments` 为空），停止计时器：`_timer_running = false`

这种机制确保了 TCP 的可靠传输：如果数据包丢失，发送方会在超时后重传，直到收到确认。

2.3. 问题三

请描述一下你为了重传未被确认的段建立的数据结构？为什么？

为了追踪已发送但尚未确认的段，我使用了一个 `std::map<uint64_t, TCPSegment>` 作为核心数据结构，键是段的起始绝对序列号，值是 TCP 段本身。

```
1 //libsponge/tcp_sender.hh
2 std::map<uint64_t, TCPSegment> _outstanding_segments{};
```

cpp

选择 `std::map` 这种数据结构的原因如下：

1. 自动排序：`std::map` 是有序容器，按照序列号自动排序。可以轻松找到序列号最小的未确认段，只需要访问 `begin()` 迭代器即可。
2. 高效查找和删除：当收到 ACK 确认时，需要快速找到并删除已被确认的段。
`std::map` 的查找和删除操作都是 $O(\log n)$ 复杂度，效率较高。
3. 键的唯一性：每个段的起始序列号是唯一的，正好适合作为 `map` 的键。
4. 便于遍历：当需要计算 `bytes_in_flight()` 时，可以方便地遍历所有未确认的段。

2.4. 问题四

请思考为什么 `TCPSender` 实现中有一个发送空段的方法，能描述一下你是怎么理解的吗？

`send_empty_segment()` 方法用于发送一个 `payload` 为空的 TCP 段。这个方法的主要用途是：

1. 发送纯 ACK 段：当接收方收到数据但不需要发送数据时，可以发送一个只包含 ACK 标志的段来确认收到的数据。虽然 TCPSender 本身不设置 ACK 标志（这由 TCPConnection 层完成），但空段为上层提供了发送 ACK 的载体。
2. 窗口更新：当接收方的窗口大小发生变化时，即使没有数据要发送，也需要发送一个段来通知发送方新的窗口大小。空段可以携带这个信息。
3. 保持连接活跃：在某些情况下，可能需要发送空段来保持 TCP 连接的活跃状态。
4. 测试和调试：在测试场景中，可能需要发送空段来验证 TCP 协议的各种行为。

在我的实现中，`send_empty_segment()` 简单地创建一个空的 TCP 段，设置正确的序列号（使用 `wrap(_next_seqno, _isn)`），然后直接推入 `_segments_out` 队列。这个段不会被加入 `_outstanding_segments`，因为它不占用序列号空间（没有 SYN、FIN 或 payload），也不需要被确认。

六、讨论、心得

通过本次实验，我深入理解了 TCP 协议的核心机制。在实现 `wrapping_integers` 时，我学会了如何处理 32 位循环序列号和 64 位绝对序列号之间的转换，这是 TCP 协议实现的基础。在实现 `TCPReceiver` 时，我理解了 TCP 接收方如何接收乱序到达的段、重组字节流，以及如何计算确认号和窗口大小来反馈给发送方。在实现 `TCPSender` 时，我亲手实现了 TCP 的可靠传输机制：窗口控制、超时重传、指数退避等。

本次实验中最困难的部分是 `unwrap` 函数的实现。由于 32 位序列号会循环，需要找到最接近 checkpoint 的绝对序列号。我发现可以利用 checkpoint 的高 32 位直接确定周期，然后根据距离判断是否需要调整到相邻周期，这样实现更简洁高效。

另一个难点是 TCP 发送方的状态管理。需要正确维护已发送但未确认的段、重传计时器、RTO 等状态，确保在各种情况下（正常发送、收到 ACK、超时重传等）都能正确工作。在实现 Sender 时，需要处理多种状态（如 Timer 运行时收到 ACK、窗口为 0 时的探测等），我体会到了 TCP 协议作为协议的复杂性。

七、附录

1. `libsponge/wrapping_integers.cc`

```
libsponge/wrapping_integers.cc
1  #include "wrapping_integers.hh"
2
3  using namespace std;
4
```

cpp

```

5  //! Transform an "absolute" 64-bit sequence number (zero-indexed) into a
   WrappingInt32
6  //! \param n The input absolute 64-bit sequence number
7  //! \param isn The initial sequence number
8  WrappingInt32 wrap(uint64_t n, WrappingInt32 isn) {
9      // 将绝对序列号转换为32位循环序列号
10     // 公式: seqno = (absolute_seqno + isn) mod 2^32
11     return WrappingInt32{static_cast<uint32_t>(n) + isn.raw_value()};
12 }
13
14 //! Transform a WrappingInt32 into an "absolute" 64-bit sequence number
   (zero-indexed)
15 //! \param n The relative sequence number
16 //! \param isn The initial sequence number
17 //! \param checkpoint A recent absolute 64-bit sequence number
18 //! \returns the 64-bit sequence number that wraps to `n` and is closest
   to `checkpoint`
19 //!
20 //! \note Each of the two streams of the TCP connection has its own ISN.
   One stream
21 //! runs from the local TCPSender to the remote TCPReceiver and has one
   ISN,
22 //! and the other stream runs from the remote TCPSender to the local
   TCPReceiver and
23 //! has a different ISN.
24 uint64_t unwrap(WrappingInt32 n, WrappingInt32 isn, uint64_t checkpoint)
   {
25     // n 在 32 位空间内相对于 isn 的偏移量
26     uint32_t offset = n.raw_value() - isn.raw_value();
27
28     // checkpoint 的高 32 位截取出来, 和 offset 拼接
29     uint64_t t = (checkpoint & 0xFFFFFFFF00000000) | offset;
30
31     // 需要判断 t 是否离 checkpoint 足够近, 是否需要 +/- 2^32
32     uint64_t shift = 1UL << 32; // 2^32
33
34     // 如果 t 比 checkpoint 大, 且差距超过了半个周期
35     if (t > checkpoint) {
36         if (t - checkpoint > (shift >> 1)) {
37             // 尝试减去 2^32, 但要确保不会变成负数 (uint64下溢)
38             if (t >= shift) {
39                 return t - shift;
40             }

```

```

41         // 如果 t < 2^32, 虽然它离 checkpoint 远, 但减去后变负数了, 只能返回
           t
42         return t;
43     }
44 } else {
45     // 如果 t 比 checkpoint 小, 且差距超过了半个周期
46     if (checkpoint - t > (shift >> 1)) {
47         // 加上 2^32, 得到下一个周期的对应值
48         return t + shift;
49     }
50 }
51
52 return t;
53 }

```

2. libsponge/tcp_receiver.hh

```

libsponge/tcp_receiver.hh cpp
1  #ifndef SPONGE_LIBSPONGE_TCP_RECEIVER_HH
2  #define SPONGE_LIBSPONGE_TCP_RECEIVER_HH
3
4  #include "byte_stream.hh"
5  #include "stream_reassembler.hh"
6  #include "tcp_segment.hh"
7  #include "wrapping_integers.hh"
8
9  #include <optional>
10
11  //! \brief The "receiver" part of a TCP implementation.
12
13  //! Receives and reassembles segments into a ByteStream, and computes
14  //! the acknowledgment number and window size to advertise back to the
15  //! remote TCPSender.
16  class TCPReceiver {
17     //! Our data structure for re-assembling bytes.
18     StreamReassembler _reassembler;
19
20     //! The maximum number of bytes we'll store.
21     size_t _capacity;
22
23     //! The initial sequence number (ISN) from the remote sender
24     std::optional<WrappingInt32> _isn{};
25

```

```

26     //!< Whether we have received FIN
27     bool _fin_received{false};
28
29     public:
30     //!< \brief Construct a TCP receiver
31     //!<
32     //!< \param capacity the maximum number of bytes that the receiver
33     //!<                    store in its buffers at any give time.
34     TCPReceiver(const size_t capacity) : _reassembler(capacity),
35     _capacity(capacity) {}
36
37     //!< \name Accessors to provide feedback to the remote TCPSender
38     //!< @{
39     //!< \brief The ackno that should be sent to the peer
40     //!< \returns empty if no SYN has been received
41     //!<
42     //!< This is the beginning of the receiver's window, or in other
43     //!< words, the sequence number
44     //!< of the first byte in the stream that the receiver hasn't
45     //!< received.
46     std::optional<WrappingInt32> ackno() const;
47
48     //!< \brief The window size that should be sent to the peer
49     //!<
50     //!< Operationally: the capacity minus the number of bytes that the
51     //!< TCPReceiver is holding in its byte stream (those that have been
52     //!< reassembled, but not consumed).
53     //!<
54     //!< Formally: the difference between (a) the sequence number of
55     //!< the first byte that falls after the window (and will not be
56     //!< accepted by the receiver) and (b) the sequence number of the
57     //!< beginning of the window (the ackno).
58     size_t window_size() const;
59     //!< @}
60
61     //!< \brief number of bytes stored but not yet reassembled
62     size_t unassembled_bytes() const { return
63     _reassembler.unassembled_bytes(); }
64
65     //!< \brief handle an inbound segment
66     void segment_received(const TCPSegment &seg);

```

```

64
65     //! \name "Output" interface for the reader
66     //!@{
67     ByteStream &stream_out() { return _reassembler.stream_out(); }
68     const ByteStream &stream_out() const { return
        _reassembler.stream_out(); }
69     //!@}
70 };
71
72 #endif // SPONGE_LIBSPONGE_TCP_RECEIVER_HH

```

3. libsponge/tcp_receiver.cc

libsponge/tcp_receiver.cc		cpp
1	#include "tcp_receiver.hh"	
2		
3	using namespace std;	
4		
5	void TCPReceiver::segment_received(const TCPSegment &seg) {	
6	// 判断当前是否为Listen状态（未正式建立连接，需要 syn 启动）	
7	if (!_isn.has_value()) {	
8	// 是，则判断是否为 SYN 包	
9	if (seg.header().syn) {	
10	// 是，获得初始序列号ISN，设置状态为SYN_RECV	
11	_isn = seg.header().seqno;	
12	} else {	
13	// 否，则数据传输未开始，丢弃数据包	
14	return;	
15	}	
16	}	
17		
18	// 否，则将该数据包中payload的数据放进流重组器（调用 push_substring），并传入	
	FIN	
19	// 注意：本实验中 SYN 和 FIN 可同时置 1	
20	// SYN 包和 FIN 包可以同时携带有具体传递的信息	
21		
22	// 计算流索引：将序列号转换为绝对序列号，然后转换为流索引	
23	// 绝对序列号从0开始（ISN对应绝对序列号0），包含SYN和FIN	
24	// 流索引从0开始，排除SYN和FIN	
25	// 对于SYN包：seqno=ISN对应absolute_seqno=0，payload从absolute_seqno=1开	
	始（stream_index=0）	
26	// 对于普通包：seqno对应absolute_seqno，数据从absolute_seqno开始	
	（stream_index=absolute_seqno-1）	

```

27 // checkpoint应该是下一个期望接收的绝对序列号
28 uint64_t checkpoint = _reassembler.stream_out().bytes_written() +
29 1; // 使用已写入字节数+1作为checkpoint (对应下一个期望的绝对序列号)
30
31 uint64_t absolute_seqno = unwrap(seg.header().seqno, _isn.value(),
32 checkpoint);
33
34 // 计算流索引
35 // 绝对序列号0是SYN, 绝对序列号1是第一个数据字节 (流索引0)
36 // 对于SYN包: seqno=ISN对应absolute_seqno=0, payload从absolute_seqno=1开始
37 // 开始 (stream_index=0)
38 // 对于普通包: seqno对应absolute_seqno, 数据从absolute_seqno开始
39 // (stream_index=absolute_seqno-1)
40 uint64_t stream_index;
41 if (seg.header().syn) {
42     // SYN包的seqno对应absolute_seqno=0, payload从absolute_seqno=1开始
43     // 所以stream_index = absolute_seqno + 1 - 1 = absolute_seqno
44     // 但实际上, 如果absolute_seqno=0, payload从stream_index=0开始
45     stream_index = absolute_seqno; // 当absolute_seqno=0时, payload从
46     stream_index=0开始
47 } else {
48     // 普通包的seqno对应absolute_seqno, 数据从absolute_seqno开始
49     // 但stream_index需要排除SYN, 所以减1
50     stream_index = absolute_seqno - 1;
51 }
52
53 // 获取payload数据
54 string payload = seg.payload().copy();
55
56 // 调用流重组器
57 _reassembler.push_substring(payload, stream_index, seg.header().fin);
58
59 // 如果收到FIN, 标记
60 if (seg.header().fin) {
61     _fin_received = true;
62 }
63
64 }
65
66 optional<WrappingInt32> TCPReceiver::ackno() const {
67     // 判断当前是否为Listen状态
68     if (!_isn.has_value()) {
69         // 是, 返回空
70         return {};
71     }
72 }

```



```

65
66     // 否, 查询尚未获取到的第一个字节的流索引, 将流索引转换为序列号 (32位的) 返回
67     // 流索引是下一个期望接收的字节的索引
68     uint64_t stream_index = _reassembler.stream_out().bytes_written();
69
70     // 将流索引转换为绝对序列号 (需要加上SYN占用的1个序列号)
71     uint64_t absolute_seqno = stream_index + 1;
72
73     // !!! 注意: 如果当前处于 FIN_RECV 状态, 则还需要加上 FIN 标志长度
74     if (_fin_received && _reassembler.stream_out().input_ended()) {
75         absolute_seqno += 1; // FIN也占用一个序列号
76     }
77
78     // 将绝对序列号转换为32位序列号
79     return wrap(absolute_seqno, _isn.value());
80 }
81
82 size_t TCPReceiver::window_size() const {
83     // 计算总容量_capacity与流重组器_reassembler中已存数据量(_output)的差值
84     size_t bytes_in_buffer = _reassembler.stream_out().bytes_written() -
85                             _reassembler.stream_out().bytes_read();
86     return _capacity - bytes_in_buffer;
87 }

```

4. libsponge/tcp_sender.hh

libsponge/tcp_sender.hh		cpp
1	#ifndef SPONGE_LIBSPONGE_TCP_SENDER_HH	
2	#define SPONGE_LIBSPONGE_TCP_SENDER_HH	
3		
4	#include "byte_stream.hh"	
5	#include "tcp_config.hh"	
6	#include "tcp_segment.hh"	
7	#include "wrapping_integers.hh"	
8		
9	#include <functional>	
10	#include <queue>	
11	#include <map>	
12		
13	/// \brief The "sender" part of a TCP implementation.	
14		
15	/// Accepts a ByteStream, divides it up into segments and sends the	
16	/// segments, keeps track of which segments are still in-flight,	

```

17  //! maintains the Retransmission Timer, and retransmits in-flight
18  //! segments if the retransmission timer expires.
19  class TCPSender {
20      private:
21          //! our initial sequence number, the number for our SYN.
22          WrappingInt32 _isn;
23
24          //! outbound queue of segments that the TCPSender wants sent
25          std::queue<TCPSegment> _segments_out{};
26
27          //! retransmission timer for the connection
28          unsigned int _initial_retransmission_timeout;
29
30          //! outgoing stream of bytes that have not yet been sent
31          ByteStream _stream;
32
33          //! the (absolute) sequence number for the next byte to be sent
34          uint64_t _next_seqno{0};
35
36          //! the (absolute) sequence number that has been acknowledged
37          uint64_t _ackno{0};
38
39          //! remote window size
40          uint16_t _window_size{1};
41
42          //! segments that have been sent but not yet acknowledged
43          std::map<uint64_t, TCPSegment> _outstanding_segments{};
44
45          //! retransmission timeout (current RT0)
46          unsigned int _rto;
47
48          //! timer for retransmission
49          unsigned int _timer{0};
50
51          //! whether the timer is running
52          bool _timer_running{false};
53
54          //! consecutive retransmissions count
55          unsigned int _consecutive_retransmissions{0};
56
57          //! whether SYN has been sent
58          bool _syn_sent{false};
59

```

```

60     //!< whether FIN has been sent
61     bool _fin_sent{false};
62
63     public:
64         //!< Initialize a TCPSender
65         TCPSender(const size_t capacity = TCPConfig::DEFAULT_CAPACITY,
66                  const uint16_t retx_timeout = TCPConfig::TIMEOUT_DFLT,
67                  const std::optional<WrappingInt32> fixed_isn = {});
68
69         //!< \name "Input" interface for the writer
70         //!<@{
71         ByteStream &stream_in() { return _stream; }
72         const ByteStream &stream_in() const { return _stream; }
73         //!<@}
74
75         //!< \name Methods that can cause the TCPSender to send a segment
76         //!<@{
77
78         //!< \brief A new acknowledgment was received
79         void ack_received(const WrappingInt32 ackno, const uint16_t
80                          window_size);
81
82         //!< \brief Generate an empty-payload segment (useful for creating
83         empty ACK segments)
84         void send_empty_segment();
85
86         //!< \brief create and send segments to fill as much of the window as
87         possible
88         void fill_window();
89
90         //!< \brief Notifies the TCPSender of the passage of time
91         void tick(const size_t ms_since_last_tick);
92         //!<@}
93
94         //!< \name Accessors
95         //!<@{
96
97         //!< \brief How many sequence numbers are occupied by segments sent
98         but not yet acknowledged?
99         //!< \note count is in "sequence space," i.e. SYN and FIN each count
100         for one byte
101         //!< (see TCPSegment::length_in_sequence_space())
102         size_t bytes_in_flight() const;

```

```

98
99     /// \brief Number of consecutive retransmissions that have occurred
    in a row
100     unsigned int consecutive_retransmissions() const;
101
102     /// \brief TCPSegments that the TCPSender has enqueued for
    transmission.
103     /// \note These must be dequeued and sent by the TCPConnection,
    /// which will need to fill in the fields that are set by the
104     TCPReceiver
105     /// (ackno and window size) before sending.
106     std::queue<TCPSegment> &segments_out() { return _segments_out; }
107     ///!@}
108
109     /// \name What is the next sequence number? (used for testing)
110     ///!@{
111
112     /// \brief absolute seqno for the next byte to be sent
113     uint64_t next_seqno_absolute() const { return _next_seqno; }
114
115     /// \brief relative seqno for the next byte to be sent
116     WrappingInt32 next_seqno() const { return wrap(_next_seqno, _isn); }
117     ///!@}
118 };
119
120 #endif // SPONGE_LIBSPONGE_TCP_SENDER_HH

```

5. libsponge/tcp_sender.cc

libsponge/tcp_sender.cc		cpp
1	#include "tcp_sender.hh"	
2		
3	#include "tcp_config.hh"	
4		
5	#include <random>	
6		
7	using namespace std;	
8		
9	/// \param[in] capacity the capacity of the outgoing byte stream	
10	/// \param[in] retx_timeout the initial amount of time to wait before retransmitting the oldest outstanding segment	
11	/// \param[in] fixed_isn the Initial Sequence Number to use, if set (otherwise uses a random ISN)	

```

12  TCPSender::TCPSender(const size_t capacity, const uint16_t retx_timeout,
13                      const std::optional<WrappingInt32> fixed_isn)
14      : _isn(fixed_isn.value_or(WrappingInt32{random_device()}()))
15      , _initial_retransmission_timeout{retx_timeout}
16      , _stream(capacity)
17      , _ackno(0)
18      , _window_size(1)
19      , _rto(retx_timeout)
20      , _timer(0)
21      , _timer_running(false)
22      , _consecutive_retransmissions(0)
23      , _syn_sent(false)
24      , _fin_sent(false) {}
25
26  uint64_t TCPSender::bytes_in_flight() const {
27      size_t total = 0;
28      for (const auto &pair : _outstanding_segments) {
29          total += pair.second.length_in_sequence_space();
30      }
31      return total;
32  }
33
34  void TCPSender::fill_window() {
35      // 如果远程窗口大小为 0，则把其视为 1 进行操作
36      uint16_t effective_window = _window_size == 0 ? 1 : _window_size;
37
38      // 计算可用窗口大小：窗口右边界 - 下一个要发送的序列号
39      // 窗口右边界 = _ackno + effective_window
40      uint64_t window_right = _ackno + effective_window;
41
42      while (_next_seqno < window_right) {
43          // 1. 如果尚未发送SYN数据包，则设置header的syn位
44          if (!_syn_sent) {
45              TCPSegment seg;
46              seg.header().syn = true;
47              seg.header().seqno = wrap(_next_seqno, _isn);
48              _syn_sent = true;
49
50              // 如果没有正在等待的数据包，则重设更新时间
51              if (_outstanding_segments.empty()) {
52                  _timer = 0;
53                  _timer_running = true;
54              }
55          }
56      }
57  }

```

```

54
55         // 发送数据包并追踪
56         uint64_t seg_start = _next_seqno;
57         _next_seqno += 1;
58         _outstanding_segments[seg_start] = seg;
59         _segments_out.push(seg);
60         continue;
61     }
62
63     // 2. 设置seqno和payload
64     // 计算可以发送的payload大小
65     // 可用空间 = window_right - _next_seqno
66     size_t available = window_right - _next_seqno;
67     size_t payload_size = min(available,
68         static_cast<size_t>(TCPConfig::MAX_PAYLOAD_SIZE));
69     payload_size = min(payload_size, _stream.buffer_size());
70
71     // 如果没有任何数据（没有设置syn，没有fin，没有payload），则停止数据包的发送（break）
72     if (payload_size == 0 && _stream.eof() && !_fin_sent) {
73         // 3. 若满足条件则增加FIN
74         // 从来没发送过 FIN 且 输入字节流处于 EOF 且 window减去payload大小
75         // 后，仍可存放下 FIN
76         if (_next_seqno < window_right) {
77             TCPSegment seg;
78             seg.header().seqno = wrap(_next_seqno, _isn);
79             seg.header().fin = true;
80             _fin_sent = true;
81
82             // 如果没有正在等待的数据包，则重设更新时间
83             if (_outstanding_segments.empty()) {
84                 _timer = 0;
85                 _timer_running = true;
86             }
87
88             // 发送数据包并追踪
89             uint64_t seg_start = _next_seqno;
90             _next_seqno += 1;
91             _outstanding_segments[seg_start] = seg;
92             _segments_out.push(seg);
93
94             // 如果设置了 fin，则break
95             break;

```

```

94         } else {
95             break;
96         }
97     }
98
99     if (payload_size == 0) {
100         break;
101     }
102
103     TCPSegment seg;
104     seg.header().seqno = wrap(_next_seqno, _isn);
105     string payload = _stream.read(payload_size);
106     seg.payload() = Buffer(move(payload));
107
108     // 3. 若满足条件则增加FIN
109     // 从来没发送过 FIN 且 输入字节流处于 EOF 且 window减去payload大小后,
110     // 仍可存放下 FIN
111     // 如果该包设置了syn的话, 还得算上syn的大小
112     if (!_fin_sent && _stream.eof()) {
113         // 检查是否有空间放置FIN
114         size_t current_seg_size = seg.length_in_sequence_space();
115         if (_next_seqno + current_seg_size < window_right) {
116             seg.header().fin = true;
117             _fin_sent = true;
118         }
119     }
120
121     size_t seg_size = seg.length_in_sequence_space();
122
123     // 如果没有正在等待的数据包, 则重设更新时间
124     if (_outstanding_segments.empty()) {
125         _timer = 0;
126         _timer_running = true;
127     }
128
129     // 发送数据包并追踪
130     uint64_t seg_start = _next_seqno;
131     _next_seqno += seg_size;
132     _outstanding_segments[seg_start] = seg;
133     _segments_out.push(seg);
134
135     // 如果设置了 fin, 则break
136     if (seg.header().fin) {

```

```

136         break;
137     }
138 }
139 }
140
141 ///! \param ackno The remote receiver's ackno (acknowledgment number)
142 ///! \param window_size The remote receiver's advertised window size
143 void TCPSender::ack_received(const WrappingInt32 ackno, const uint16_t
window_size) {
144     // 将ackno转换为绝对序列号
145     uint64_t absolute_ackno = unwrap(ackno, _isn, _ackno);
146
147     // 如果传入的 ack 是不可靠的 (ack_seqno大于next_seqno)，则直接丢弃
148     if (absolute_ackno > _next_seqno) {
149         return;
150     }
151
152     // 更新窗口大小
153     _window_size = window_size;
154
155     // 遍历数据结构（用来存储发送的segment），如果一个发送的segment已经被成功接收，
则
156     bool acked_something = false;
157     for (auto it = _outstanding_segments.begin(); it !=
_outstanding_segments.end(); ) {
158         uint64_t seg_start = it->first;
159         uint64_t seg_end = seg_start + it-
>second.length_in_sequence_space();
160
161         if (seg_end ≤ absolute_ackno) {
162             // 1. 从数据结构中将该segment丢弃
163             it = _outstanding_segments.erase(it);
164             acked_something = true;
165         } else {
166             ++it;
167         }
168     }
169
170     if (acked_something) {
171         // 2. 重置重传计时器
172         _timer = 0;
173         // 3. 将RTO重置为初始值
174         _rto = _initial_retransmission_timeout;

```



```

175         // 重置连续重传计数器
176         _consecutive_retransmissions = 0;
177     }
178
179     // 更新_ackno
180     if (absolute_ackno > _ackno) {
181         _ackno = absolute_ackno;
182     }
183
184     // 调用fill_window继续发送数据（更新窗口大小后，接收方可能有了新的接收空间，所以应该再次发送数据）
185     fill_window();
186
187     // 如果所有数据都被确认了，停止重传计时器
188     if (_outstanding_segments.empty()) {
189         _timer_running = false;
190     }
191 }
192
193 ///! \param[in] ms_since_last_tick the number of milliseconds since the
194 last call to this method
195 void TCPSender::tick(const size_t ms_since_last_tick) {
196     // 如果计时器正在运行，更新计时器
197     if (_timer_running) {
198         _timer += ms_since_last_tick;
199
200         // 如果重传计时器超时
201         if (_timer ≥ _rto) {
202             // 遍历追踪列表，如果存在发送中的数据包，并且重传计时器超时
203             if (!_outstanding_segments.empty()) {
204                 // 重置重传定时器
205                 _timer = 0;
206
207                 // 重传尚未被 TCP 接收方完全确认的序列号最小的段
208                 auto it = _outstanding_segments.begin();
209                 _segments_out.push(it→second);
210
211                 // 如果对方接收窗口大小不为0（说明网络拥堵）
212                 if (_window_size > 0) {
213                     // 超时重传时间 RTO 的值加倍，即*2
214                     _rto *= 2;
215                     // 连续重传计时器增加
216                     _consecutive_retransmissions++;

```

```

216         } else {
217             // 如果窗口大小为0，不增加RT0，但增加连续重传计数
218             _consecutive_retransmissions++;
219         }
220     }
221 }
222 }
223 }
224
225 unsigned int TCPSender::consecutive_retransmissions() const {
226     return _consecutive_retransmissions;
227 }
228
229 void TCPSender::send_empty_segment() {
230     // 生成并发送一个payload长度为零的TCPSegment，并且序列号设置正确
231     TCPSegment seg;
232     seg.header().seqno = wrap(_next_seqno, _isn);
233     _segments_out.push(seg);
234 }

```